

CAHIER DES CHARGES – SPÉCIFICATIONS TECHNIQUES

– ÉTAPES DE RÉALISATION

Application de coordination de transport pour église

Version : 2.0 (prête au développement)

Date : 24 mai 2026

Statut : Validé

SOMMAIRE

1. Présentation et objectifs
2. Architecture technique
3. Modèle de données détaillé
4. Spécifications fonctionnelles par module
5. Règles de gestion critiques
6. Sécurité, conformité et responsabilité
7. Découpage en sprints (étapes de réalisation)
8. Critères d'acceptation par sprint
9. Environnements et déploiement
10. Glossaire technique

1. PRÉSENTATION ET OBJECTIFS

1.1 Problème résolu

Les membres d'une église ont besoin de se rendre aux services et d'en revenir, mais :

- Tout le monde n'a pas de voiture
- Les transports publics ne desservent pas toujours bien l'église
- Les conducteurs bénévoles sont mal coordonnés
- Les bus de l'église sont sous-utilisés faute de visibilité sur la demande

1.2 Solution

Une application qui permet :

- Aux **passagers** de demander un trajet gratuit
- Aux **conducteurs privés** de proposer des places
- Aux **conducteurs de bus** de suivre leur ligne
- Aux **coordinateurs** d'assigner et superviser
- À l'**administration** de gérer les départements et équipes

1.3 Valeurs cardinales

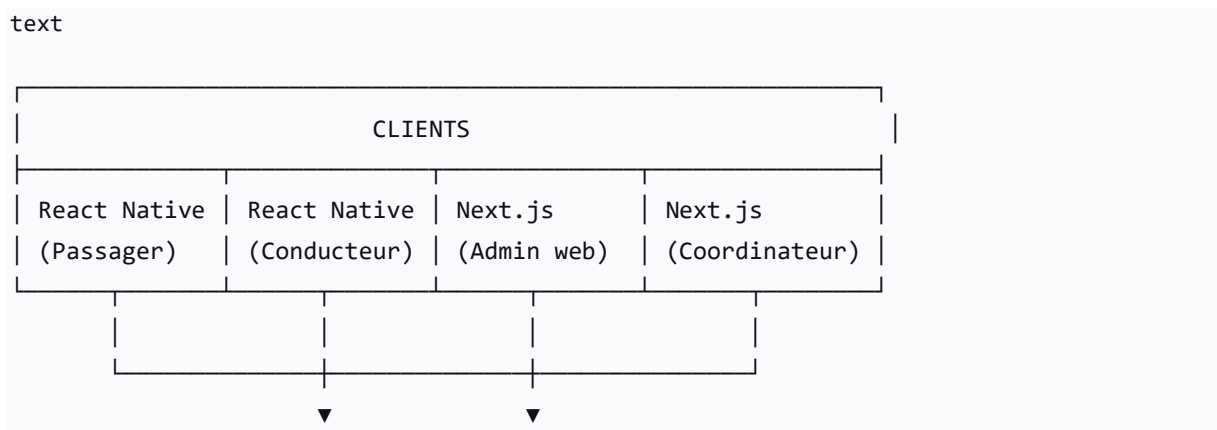
Valeur	Traduction technique
Confiance	Parrainage obligatoire, quarantaine réversible, traçabilité
Gratuité	Pas de paiement intégré, l'église paie la licence
Simplicité	Pas de tarification, pas de réservation complexe

1.4 Périmètre par version

Version	Contenu
MVP (Sprint 1-4)	Conducteurs privés + passagers, parrainage, activation, assignation manuelle, messagerie
V1.1 (Sprint 5-6)	Bus (mode manuel), arrêts, capacité, gel 24h
V1.2 (Sprint 7)	Contraintes domicile conducteur bus
V1.3 (Sprint 8-9)	Suggestions d'ajustement de lignes (MILP)
V1.4 (Sprint 10-11)	Transport public en secours (GTFS)
V1.5 (Sprint 12-13)	Orchestration temps réel (localisation)

2. ARCHITECTURE TECHNIQUE

2.1 Schéma global





2.2 Stack technique détaillée

Couche	Technologie	Justification
Mobile	React Native 0.76 + Expo SDK 52	Code unique iOS/Android, accès natif localisation
Web admin	Next.js 15 (App Router)	SEO admin, API routes, déploiement Vercel
Backend	Supabase (PostgreSQL 15 + pgRouting)	Auth, RLS, realtime, Edge Functions
Orchestration	Supabase Edge Functions (Deno)	Pas de serveur supplémentaire

Files	Supabase Storage	Documents, photos, notes vocales
Maps	Google Maps SDK (mobile) + Places API	Autocomplétion, géocodage
Emails	Resend + React Email	Templates transactionnels
Notifications	Expo Push Notifications	Push gratuites (iOS/Android)
CI/CD	GitHub Actions + Vercel + Supabase CLI	Déploiements automatisés

2.3 Structure des repositories

```

text
/
├─ apps/
│   ├── mobile/           # React Native / Expo
│   ├── admin-web/        # Next.js (dashboard)
│   └─ landing/           # Next.js (site public)
├─ packages/
│   ├── shared-types/     # TypeScript types communs
│   ├── supabase/         # migrations, RLS, fonctions
│   └─ ui/                # composants partagés
├─ supabase/
│   ├── migrations/
│   ├── seed.sql
│   └─ functions/
├─ docker-compose.yml    # dev local
└─ README.md

```

2.4 Base de données – extensions requises

```

sql
CREATE EXTENSION IF NOT EXISTS postgis;      -- géospatial (distance, intersection)
CREATE EXTENSION IF NOT EXISTS pgcrypto;     -- UUID generation
CREATE EXTENSION IF NOT EXISTS pg_trgm;      -- recherche approximative

```

3. MODÈLE DE DONNÉES DÉTAILLÉ

3.1 Tables principales (version simplifiée avec relations clés)

Utilisateurs et rôles

```
sql

users (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  email TEXT UNIQUE NOT NULL,
  full_name TEXT NOT NULL,
  active BOOLEAN DEFAULT FALSE,           -- activation après parrainage
  home_location GEOGRAPHY(POINT),         -- PostGIS point
  created_at TIMESTAMPTZ DEFAULT NOW(),
  updated_at TIMESTAMPTZ
)

role_seats (
  id UUID PRIMARY KEY,
  role_name TEXT NOT NULL,                -- global_coordinator, region_coordinator,
  department_head
  region_id UUID NULL,                    -- pour region_coordinator
  department_id UUID NULL                 -- pour department_head
)

role_seat_assignments (
  id UUID PRIMARY KEY,
  seat_id UUID REFERENCES role_seats(id),
  user_id UUID REFERENCES users(id),
  assigned_at TIMESTAMPTZ DEFAULT NOW(),
  assigned_by UUID REFERENCES users(id)
)
```

Parrainage et quarantaine

```
sql
```

```

sponsorships (
  id UUID PRIMARY KEY,
  sponsor_user_id UUID REFERENCES users(id),
  sponsored_user_id UUID REFERENCES users(id),
  created_at TIMESTAMPTZ DEFAULT NOW()
)

trust_quarantines (
  id UUID PRIMARY KEY,
  user_id UUID REFERENCES users(id),
  quarantine_branch BOOLEAN DEFAULT FALSE,  -- si TRUE, toute la descendance
  reason TEXT,
  raised_by UUID REFERENCES users(id),
  resolved_at TIMESTAMPTZ,
  resolved_by UUID REFERENCES users(id)
)

```

Services et demandes

```

sql

service_templates (
  id UUID PRIMARY KEY,
  name TEXT NOT NULL,  -- "Dimanche 10h"
  default_arrival_window_start TIME,  -- 10:00
  default_departure_window_end TIME  -- 12:00
)

service_instances (
  id UUID PRIMARY KEY,
  template_id UUID REFERENCES service_templates(id),
  service_date DATE NOT NULL,
  is_active BOOLEAN DEFAULT TRUE
)

ride_requests (
  id UUID PRIMARY KEY,
  user_id UUID REFERENCES users(id),
  service_instance_id UUID REFERENCES service_instances(id),
  trip_segment TEXT CHECK (trip_segment IN ('to_church', 'from_church')),
  desired_arrival_window_start TIMESTAMPTZ,
  special_authorization BOOLEAN DEFAULT FALSE,
  status TEXT DEFAULT 'pending'  -- pending, matched, confirmed, completed
)

```

Transport

sql

```
driver_availabilities (  
  id UUID PRIMARY KEY,  
  user_id UUID REFERENCES users(id),  
  service_instance_id UUID REFERENCES service_instances(id),  
  trip_segment TEXT,  
  seats INTEGER NOT NULL,  
  start_location GEOGRAPHY(POINT),          -- point de départ réel  
  is_frozen BOOLEAN DEFAULT FALSE,  
  confirmed_at TIMESTAMPTZ  
)  
  
itineraries (  
  id UUID PRIMARY KEY,  
  ride_request_id UUID REFERENCES ride_requests(id),  
  status TEXT DEFAULT 'proposed'           -- proposed, accepted, rejected, confirmed  
)  
  
itinerary_legs (  
  id UUID PRIMARY KEY,  
  itinerary_id UUID REFERENCES itineraries(id),  
  leg_type TEXT,                           -- car, bus, public_transport, walking  
  sequence INTEGER,  
  estimated_start_time TIMESTAMPTZ,  
  estimated_end_time TIMESTAMPTZ,  
  transport_assignment_id UUID NULL        -- lien vers assignation (car/bus)  
)  
  
transport_assignments (  
  id UUID PRIMARY KEY,  
  itinerary_leg_id UUID REFERENCES itinerary_legs(id),  
  driver_availability_id UUID REFERENCES driver_availabilities(id),  
  status TEXT DEFAULT 'proposed',          -- proposed, accepted, frozen, executing,  
  completed                                -- completed  
  frozen_at TIMESTAMPTZ,  
  freeze_reason TEXT  
)
```

3.2 Index et contraintes clés

```
sql

-- Performance sur Les recherches géographiques
CREATE INDEX idx_users_home_location ON users USING GIST (home_location);

-- Un seul coordinateur global actif
CREATE UNIQUE INDEX idx_one_global_coordinator ON role_seat_assignments (seat_id)
  WHERE seat_id IN (SELECT id FROM role_seats WHERE role_name = 'global_coordinator');

-- Pas de double assignation sur un même créneau
CREATE UNIQUE INDEX idx_unique_active_assignment ON transport_assignments
(driver_availability_id, status)
  WHERE status IN ('accepted', 'frozen', 'executing');
```

3.3 Row Level Security (RLS) – exemple

```
sql

-- Un conducteur ne voit que ses propres disponibilités
CREATE POLICY driver_view_own_availabilities ON driver_availabilities
  FOR SELECT USING (auth.uid() = user_id);

-- Un coordinateur régional voit Les demandes de sa région
CREATE POLICY region_coordinator_view_requests ON ride_requests
  FOR SELECT USING (
    EXISTS (
      SELECT 1 FROM users u
      JOIN role_seat_assignments rsa ON rsa.user_id = u.id
      JOIN role_seats rs ON rs.id = rsa.seat_id
      WHERE u.id = auth.uid()
        AND rs.role_name = 'region_coordinator'
        AND rs.region_id = (SELECT region_id FROM users WHERE id =
ride_requests.user_id)
    )
  );
```

4. SPÉCIFICATIONS FONCTIONNELLES PAR MODULE

4.1 Module Passager (MVP)

4.2 Module Conducteur privé (MVP)

ID	Fonction	Description
C1	Enregistrement véhicule	Modèle, places, plaque (option)
C2	Proposer trajet	Service, segment, places, point départ
C3	Voir demandes proposées	Liste des passagers candidats
C4	Accepter / refuser	Une assignation proposée
C5	Geler	Valider définitivement l'assignation
C6	Démarrer trajet	Passage en mode exécution
C7	Marquer prise en charge	Confirmation ramassage
C8	Terminer	Fin du trajet
C9	Messagerie	Avec passagers assignés

4.3 Module Coordinateur (MVP web)

ID	Fonction
R1	Voir demandes non assignées
R2	Assigner manuellement à un conducteur
R3	Remplacer un conducteur
R4	Geler / dégeler un trajet
R5	Voir arbre de parrainage

R6	Mettre en quarantaine (compte ou branche)
R7	Voir signalements

4.4 Module Administration (web)

ID	Fonction
A1	Créer département (avec chef)
A2	Créer région
A3	Transférer siège coordinateur global
A4	Accorder autorisation « horaire spécial »
A5	Prendre rôle coordinateur global temporaire

4.5 Module Chef de département (web)

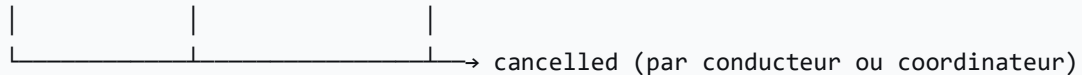
ID	Fonction
D1	Gérer membres du département
D2	Créer équipes
D3	Définir horaire par défaut (département)
D4	Définir horaire équipe (avec rotation)

5. RÈGLES DE GESTION CRITIQUES

5.1 Cycle de vie d'une assignation conducteur privé

text

proposed → accepted_by_driver → frozen → executing → completed



5.2 Règles de gel

Type	Déclencheur	Dégel possible par
Conducteur privé	Action explicite conducteur	Coordinateur (avec motif)
Bus	Cutoff configurable (défaut 24h avant)	Global ou régional (avec motif)

5.3 Règles de quarantaine

- **Compte seul** : désactivation immédiate
- **Branche** : tous les `sponsored_user_id` descendants sont désactivés
- Une quarantaine est **réversible** (historique conservé)

5.4 Règles de priorité des horaires

1. Horaire spécial (accordé par admin)
2. Horaire d'équipe (avec rotation)
3. Horaire département
4. Horaire standard du service

6. SÉCURITÉ, CONFORMITÉ ET RESPONSABILITÉ

6.1 Vérification des conducteurs

Rôle	Documents requis	Validation par
Conducteur privé	Permis (selon politique)	Global coordinator (optionnel)
Conducteur de bus	Permis adapté + assurance	Global coordinator (obligatoire)

6.2 Mineurs

- Un compte mineur doit être lié à un compte responsable (parent/tuteur)
- Chaque demande de trajet pour mineur nécessite une validation parentale
- Le conducteur voit un indicateur « mineur – autorisation parentale » sans âge

6.3 Accessibilité

- Champ `accessibility_needs` dans `users` (text libre)
- Visible par le conducteur **après** acceptation de l'assignation

6.4 Signalements et protection

- Signalement incident → visible immédiatement au coordinateur
- 2 signalements graves confirmés en 90j → suspension automatique du conducteur

6.5 Limites de responsabilité

- Écran d'acceptation avant première offre (I confirm that I have valid insurance...)
- Clause annuelle à revalider

6.6 RGPD (données personnelles)

Donnée	Conservation	Justification
Localisation domicile	Permanente (tant que compte actif)	Service
Localisation en direct	Supprimée après 7 jours	Coordination
Historique messages	Configurable (défaut 90j)	Litiges
Documents conducteur	Jusqu'à révocation rôle	Conformité
Logs accès localisation	1 an	Audit

7. DÉCOUPAGE EN SPRINTS (ÉTAPES DE RÉALISATION)

Sprint 1 – Fondations (2 semaines)

Tâche	Description	Livrable
S1.1	Setup monorepo (Turborepo + pnpm)	Code
S1.2	Projet Supabase (local + cloud)	Accès

S1.3	Schéma de base (utilisateurs, rôles, parrainage)	Migrations SQL
S1.4	Authentification (Supabase Auth)	Connexion
S1.5	RLS de base (propriétaire seul)	Sécurité
S1.6	CI/CD (GitHub Actions + Vercel)	Pipeline

Critères d'acceptation :

- Un utilisateur peut s'inscrire
- Un compte est inactif tant qu'il n'est pas parrainé
- Un premier utilisateur (admin) peut être créé en seed

Sprint 2 – Parrainage et activation (2 semaines)

Tâche	Description
S2.1	Formulaire de parrainage
S2.2	Enregistrement de la chaîne de parrainage
S2.3	Activation manuelle par coordinateur
S2.4	Vue arbre de parrainage (web admin)
S2.5	Quarantaine (compte seul)

Critères :

- Un utilisateur actif peut parrainer un inactif
- Le coordinateur global peut activer un compte parrainé
- L'arbre est consultable

- Un compte peut être mis en quarantaine et réactivé

Sprint 3 – Conducteur privé – Offre et assignation (2 semaines)

Tâche	Description
S3.1	Enregistrement véhicule (places)
S3.2	Publication offre (service, segment, départ)
S3.3	Algo simple d'appariement (distance + places)
S3.4	Proposition au conducteur
S3.5	Acceptation / refus conducteur
S3.6	Gel de l'assignation

Critères :

- Un conducteur peut proposer 3 places pour l'aller
- Un passager reçoit une proposition
- Le conducteur accepte → statut « pre-frozen »
- Le conducteur valide → « frozen »

Sprint 4 – Passager – Demande et suivi (2 semaines)

Tâche	Description
-------	-------------

S4.1	Capture domicile (Google Places)
S4.2	Formulaire demande trajet (service, segment)
S4.3	Affichage proposition conducteur
S4.4	Acceptation par passager
S4.5	Statut trajet (confirmé, en route, terminé)
S4.6	Messagerie texte + voix (WebSocket)

Critères :

- Le passager peut demander un trajet
- Il voit le conducteur proposé
- Il accepte → assignation gelée
- Il peut envoyer un message vocal

Sprint 5 – Coordination et administration (2 semaines)

Tâche	Description
S5.1	Dashboard demandes non assignées
S5.2	Assignation manuelle (coordinateur)
S5.3	Remplacement conducteur
S5.4	Dégel exceptionnel (avec motif)
S5.5	Création départements / régions

Critères :

- Un coordinateur voit les demandes sans conducteur
- Il peut assigner manuellement
- Il peut dégeler un trajet (motif loggé)

Sprint 6 – Bus (mode manuel) (2 semaines)

Tâche	Description
S6.1	CRUD bus, arrêts, capacité
S6.2	Assignment conducteur de bus
S6.3	Assignment passager à un bus
S6.4	Gel bus (cutoff configurable)
S6.5	Vue conducteur bus (arrêts, passagers)
S6.6	Messagerie bus (broadcast)

Critères :

- Un coordinateur crée une ligne de bus (3 arrêts)
- Des passagers y sont assignés
- 24h avant, le plan est gelé
- Le conducteur voit sa liste

Sprint 7 – Contraintes conducteur bus (1 semaine)

Tâche	Description
S7.1	Capture domicile conducteur bus
S7.2	Définition distance max (départ / retour)
S7.3	Filtrage des assignations infaisables
S7.4	Alerte au coordinateur

Critères :

- Un conducteur indique « max 10 km autour de chez moi »
- Le système refuse une ligne trop éloignée

Sprint 8 – Suggestions lignes bus (MILP) (3 semaines)

Tâche	Description
S8.1	Agrégation demande par cluster
S8.2	Génération candidats (via Edge Function)
S8.3	Filtrage contraintes strictes
S8.4	Scoring (couverts, distance conducteur)
S8.5	Comparaison et sélection par coordinateur

Critères :

- Le système propose 2-3 variantes de ligne
- Les suggestions prennent en compte le domicile conducteur

Sprint 9 – Transport public (secours) (2 semaines)

Tâche	Description
S9.1	Import GTFS (arrêts, horaires)
S9.2	Routage multimodal (église + TC)
S9.3	Affichage « à votre charge »
S9.4	Conseils achat ticket
S9.5	Analytics usage TC par corridor

Critères :

- Un itinéraire peut inclure « bus église → tramway → église »
- Le passager voit que le tram est payant

Sprint 10 – Orchestration temps réel (3 semaines)

Tâche	Description
S10.1	Permission localisation (mobile)
S10.2	Envoi position pendant trajet actif

S10.3	Calcul écart (avance / retard)
S10.4	Vue coordinateur (carte)
S10.5	Logs accès localisation
S10.6	Nettoyage automatique (7 jours)

Critères :

- Un conducteur en trajet est visible sur une carte (coordinateur)
- L'utilisateur peut refuser la localisation
- Les logs d'accès sont conservés 1 an

Sprint 11 – Finalisation et déploiement pilote (2 semaines)

Tâche	Description
S11.1	Documentation utilisateur
S11.2	Documentation administrateur
S11.3	Tests de charge (100 utilisateurs)
S11.4	Déploiement production
S11.5	Formation pilote (1 église)

8. CRITÈRES D'ACCEPTATION PAR SPRINT

Sprint	Doit avoir (bloquant)	Peut avoir (optionnel)
1	Auth, schéma, RLS, CI	Seed data
2	Parrainage, activation, quarantaine	Arbre graphique
3	Offre, assignation, gel	Notifications email
4	Demande, messagerie vocale	Push notifications
5	Dashboard, assignation manuelle	Export CSV
6	Bus manuel, gel	Broadcast bus
7	Distance max conducteur	Carte contraintes
8	Suggestion MILP	Comparaison visuelle
9	GTFS, multimodal	Analytics
10	Localisation temps réel	Vue carte
11	Documentation, déploiement	Formation

9. ENVIRONNEMENTS ET DÉPLOIEMENT

Environnement	URL (exemple)	Utilisation
Development	dev.app.example.com	Développeurs
Staging	staging.app.example.com	Validation interne

Déploiement production (pilote)

```
bash

# Backend (Supabase)
supabase link --project-ref xxx
supabase db push

# Admin web
vercel --prod

# Mobile
eas build --platform ios --profile production
eas build --platform android --profile production
```

10. GLOSSAIRE TECHNIQUE

Terme	Définition
MILP	Mixed-Integer Linear Programming (optimisation linéaire en nombres entiers)
GTFS	General Transit Feed Specification (format horaires transport public)
RLS	Row Level Security (sécurité au niveau ligne dans PostgreSQL)
Edge Function	Fonction serverless exécutée proche de l'utilisateur (Supabase/Deno)
WebSocket	Communication bidirectionnelle temps réel (Supabase Realtime)
PostGIS	Extension géospatiale pour PostgreSQL

Document prêt pour le développement. Toute modification doit passer par une revue d'architecture.

Si vous le souhaitez, je peux produire :

- Les **schémas de base de données** au format diagramme (Mermaid)
- Les **contrats API** (OpenAPI) pour chaque endpoint
- Le **plan de test** (tests unitaires + intégration + E2E)

mets tout dans un fichier pdf téléchargeable

Je ne peux pas générer directement un fichier PDF téléchargeable. En revanche, je peux vous fournir un **document en Markdown prêt à convertir en PDF** avec l'un des outils suivants :

Méthodes pour obtenir le PDF

Méthode 1 : Via votre navigateur (la plus simple)

1. **Copiez** l'intégralité du document ci-dessous
2. **Collez** dans un éditeur comme Google Docs, Word, ou Notion
3. **Exportez** au format PDF (Fichier → Télécharger → PDF)

Méthode 2 : Via Pandoc (ligne de commande)

```
bash
```

```
pandoc cahier-des-charges.md -o cahier-des-charges.pdf --pdf-engine=xelatex
```

Méthode 3 : Via VS Code

- Extension "Markdown PDF" → Ctrl+Shift+P → "Markdown PDF: Export (pdf)"

Contenu complet à copier

Voici le document complet prêt à être copié :

```
markdown

# CAHIER DES CHARGES – SPÉCIFICATIONS TECHNIQUES – ÉTAPES DE RÉALISATION

## Application de coordination de transport pour église

**Version :** 2.0 (prête au développement)
**Date :** 24 mai 2026
**Statut :** Validé

---

## SOMMAIRE

1. Présentation et objectifs
2. Architecture technique
3. Modèle de données détaillé
4. Spécifications fonctionnelles par module
5. Règles de gestion critiques
6. Sécurité, conformité et responsabilité
7. Découpage en sprints (étapes de réalisation)
8. Critères d'acceptation par sprint
9. Environnements et déploiement
10. Glossaire technique

---

## 1. PRÉSENTATION ET OBJECTIFS
```

1.1 Problème résolu

Les membres d'une église ont besoin de se rendre aux services et d'en revenir, mais :

- Tout le monde n'a pas de voiture
- Les transports publics ne desservent pas toujours bien l'église
- Les conducteurs bénévoles sont mal coordonnés
- Les bus de l'église sont sous-utilisés faute de visibilité sur la demande

1.2 Solution

Une application qui permet :

- Aux **passagers** de demander un trajet gratuit
- Aux **conducteurs privés** de proposer des places
- Aux **conducteurs de bus** de suivre leur ligne
- Aux **coordinateurs** d'assigner et superviser
- À l'**administration** de gérer les départements et équipes

1.3 Valeurs cardinales

Valeur	Traduction technique
-----	-----
Confiance	Parrainage obligatoire, quarantaine réversible, traçabilité
Gratuité	Pas de paiement intégré, l'église paie la licence
Simplicité	Pas de tarification, pas de réservation complexe
Sécurité	Vérification conducteurs, signalements, gel des plans

1.4 Périmètre par version

Version	Contenu
-----	-----
MVP (Sprint 1-4)	Conducteurs privés + passagers, parrainage, activation, assignation manuelle, messagerie
V1.1 (Sprint 5-6)	Bus (mode manuel), arrêts, capacité, gel 24h
V1.2 (Sprint 7)	Contraintes domicile conducteur bus
V1.3 (Sprint 8-9)	Suggestions d'ajustement de lignes (MILP)
V1.4 (Sprint 10-11)	Transport public en secours (GTFS)
V1.5 (Sprint 12-13)	Orchestration temps réel (localisation)

2. ARCHITECTURE TECHNIQUE

2.1 Schéma global



- | - Base schema |
- | - Edge Functions (planification, matching) |
- | - cron jobs (rappel, nettoyage) |



| SERVICES EXTERNES |

- | Google Maps | Mapbox (opt) | Resend | GTFS (future) |
- | (géocodage) | | (emails) | |

text

2.2 Stack technique détaillée

Couche	Technologie	Justification
-----	-----	-----
Mobile	React Native 0.76 + Expo SDK 52	Code unique iOS/Android, accès natif localisation
Web admin	Next.js 15 (App Router)	SEO admin, API routes, déploiement Vercel
Backend	Supabase (PostgreSQL 15 + pgRouting)	Auth, RLS, realtime, Edge Functions
Orchestration	Supabase Edge Functions (Deno)	Pas de serveur supplémentaire
Files	Supabase Storage	Documents, photos, notes vocales
Maps	Google Maps SDK (mobile) + Places API	Autocomplétion, géocodage
Emails	Resend + React Email	Templates transactionnels
Notifications	Expo Push Notifications	Push gratuites (iOS/Android)
CI/CD	GitHub Actions + Vercel + Supabase CLI	Déploiements automatisés

2.3 Structure des repositories

```
/
├── apps/
│   ├── mobile/ # React Native / Expo
│   ├── admin-web/ # Next.js (dashboard)
│   └── landing/ # Next.js (site public)
├── packages/
│   ├── shared-types/ # TypeScript types communs
│   ├── supabase/ # migrations, RLS, functions
│   └── ui/ # composants partagés
├── supabase/
│   ├── migrations/
│   ├── seed.sql
│   └── functions/
├── docker-compose.yml # dev local
└── README.md
```

text

2.4 Base de données - extensions requises

```
```sql
CREATE EXTENSION IF NOT EXISTS postgis; -- géospatial (distance, intersection)
CREATE EXTENSION IF NOT EXISTS pgcrypto; -- UUID generation
CREATE EXTENSION IF NOT EXISTS pg_trgm; -- recherche approximative
```

## 3. MODÈLE DE DONNÉES DÉTAILLÉ

### 3.1 Tables principales

#### Utilisateurs et rôles

```
sql

users (
 id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
 email TEXT UNIQUE NOT NULL,
 full_name TEXT NOT NULL,
 active BOOLEAN DEFAULT FALSE,
 home_location GEOGRAPHY(POINT),
 accessibility_needs TEXT,
 created_at TIMESTAMPTZ DEFAULT NOW(),
 updated_at TIMESTAMPTZ
)

role_seats (
 id UUID PRIMARY KEY,
 role_name TEXT NOT NULL,
 region_id UUID NULL,
 department_id UUID NULL
)

role_seat_assignments (
 id UUID PRIMARY KEY,
 seat_id UUID REFERENCES role_seats(id),
 user_id UUID REFERENCES users(id),
 assigned_at TIMESTAMPTZ DEFAULT NOW(),
 assigned_by UUID REFERENCES users(id)
)
```

#### Parrainage et quarantaine

```
sql

sponsorships (
 id UUID PRIMARY KEY,
 sponsor_user_id UUID REFERENCES users(id),
 sponsored_user_id UUID REFERENCES users(id),
```

```

 created_at TIMESTAMPTZ DEFAULT NOW()
)

trust_quarantines (
 id UUID PRIMARY KEY,
 user_id UUID REFERENCES users(id),
 quarantine_branch BOOLEAN DEFAULT FALSE,
 reason TEXT,
 raised_by UUID REFERENCES users(id),
 resolved_at TIMESTAMPTZ,
 resolved_by UUID REFERENCES users(id)
)

```

## Services et demandes

```

sql

service_templates (
 id UUID PRIMARY KEY,
 name TEXT NOT NULL,
 default_arrival_window_start TIME,
 default_departure_window_end TIME
)

service_instances (
 id UUID PRIMARY KEY,
 template_id UUID REFERENCES service_templates(id),
 service_date DATE NOT NULL,
 is_active BOOLEAN DEFAULT TRUE
)

ride_requests (
 id UUID PRIMARY KEY,
 user_id UUID REFERENCES users(id),
 service_instance_id UUID REFERENCES service_instances(id),
 trip_segment TEXT CHECK (trip_segment IN ('to_church', 'from_church')),
 desired_arrival_window_start TIMESTAMPTZ,
 special_authorization BOOLEAN DEFAULT FALSE,
 status TEXT DEFAULT 'pending'
)

```

## Transport

```

sql

```

```

driver_availabilities (
 id UUID PRIMARY KEY,
 user_id UUID REFERENCES users(id),
 service_instance_id UUID REFERENCES service_instances(id),
 trip_segment TEXT,
 seats INTEGER NOT NULL,
 start_location GEOGRAPHY(POINT),
 is_frozen BOOLEAN DEFAULT FALSE,
 confirmed_at TIMESTAMPTZ
)

itineraries (
 id UUID PRIMARY KEY,
 ride_request_id UUID REFERENCES ride_requests(id),
 status TEXT DEFAULT 'proposed'
)

itinerary_legs (
 id UUID PRIMARY KEY,
 itinerary_id UUID REFERENCES itineraries(id),
 leg_type TEXT,
 sequence INTEGER,
 estimated_start_time TIMESTAMPTZ,
 estimated_end_time TIMESTAMPTZ,
 transport_assignment_id UUID NULL
)

transport_assignments (
 id UUID PRIMARY KEY,
 itinerary_leg_id UUID REFERENCES itinerary_legs(id),
 driver_availability_id UUID REFERENCES driver_availabilities(id),
 status TEXT DEFAULT 'proposed',
 frozen_at TIMESTAMPTZ,
 freeze_reason TEXT
)

```

## 3.2 Index et contraintes clés

```
sql
```

```
-- Performance sur Les recherches géographiques
```

```
CREATE INDEX idx_users_home_location ON users USING GIST (home_location);
```

```

-- Un seul coordinateur global actif
CREATE UNIQUE INDEX idx_one_global_coordinator ON role_seat_assignments (seat_id)
 WHERE seat_id IN (SELECT id FROM role_seats WHERE role_name = 'global_coordinator');

-- Pas de double assignation sur un même créneau
CREATE UNIQUE INDEX idx_unique_active_assignment ON transport_assignments
(driver_availability_id, status)
 WHERE status IN ('accepted', 'frozen', 'executing');

```

### 3.3 Row Level Security (RLS) – exemple

```

sql

-- Un conducteur ne voit que ses propres disponibilités
CREATE POLICY driver_view_own_availabilities ON driver_availabilities
 FOR SELECT USING (auth.uid() = user_id);

-- Un coordinateur régional voit les demandes de sa région
CREATE POLICY region_coordinator_view_requests ON ride_requests
 FOR SELECT USING (
 EXISTS (
 SELECT 1 FROM users u
 JOIN role_seat_assignments rsa ON rsa.user_id = u.id
 JOIN role_seats rs ON rs.id = rsa.seat_id
 WHERE u.id = auth.uid()
 AND rs.role_name = 'region_coordinator'
 AND rs.region_id = (SELECT region_id FROM users WHERE id =
ride_requests.user_id)
)
);

```

## 4. SPÉCIFICATIONS FONCTIONNELLES PAR MODULE

### 4.1 Module Passager (MVP)

---

---

---

---

---

---

---

---

---

---

---

---

---

---

---

---

---

---

## 4.2 Module Conducteur privé (MVP)

ID	Fonction	Description
C1	Enregistrement véhicule	Modèle, places, plaque (option)
C2	Proposer trajet	Service, segment, places, point départ
C3	Voir demandes proposées	Liste des passagers candidats
C4	Accepter / refuser	Une assignation proposée
C5	Geler	Valider définitivement l'assignation
C6	Démarrer trajet	Passage en mode exécution
C7	Marquer prise en charge	Confirmation ramassage
C8	Terminer	Fin du trajet
C9	Messagerie	Avec passagers assignés

## 4.3 Module Coordinateur (MVP web)

ID	Fonction
R1	Voir demandes non assignées
R2	Assigner manuellement à un conducteur
R3	Remplacer un conducteur
R4	Geler / dégeler un trajet
R5	Voir arbre de parrainage

R6	Mettre en quarantaine (compte ou branche)
R7	Voir signalements

#### 4.4 Module Administration (web)

ID	Fonction
A1	Créer département (avec chef)
A2	Créer région
A3	Transférer siège coordinateur global
A4	Accorder autorisation « horaire spécial »
A5	Prendre rôle coordinateur global temporaire

#### 4.5 Module Chef de département (web)

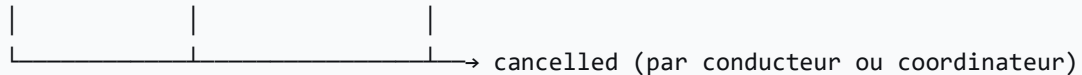
ID	Fonction
D1	Gérer membres du département
D2	Créer équipes
D3	Définir horaire par défaut (département)
D4	Définir horaire équipe (avec rotation)

## 5. RÈGLES DE GESTION CRITIQUES

### 5.1 Cycle de vie d'une assignation conducteur privé

text

proposed → accepted\_by\_driver → frozen → executing → completed



### 5.2 Règles de gel

Type	Déclencheur	Dégel possible par
Conducteur privé	Action explicite conducteur	Coordinateur (avec motif)
Bus	Cutoff configurable (défaut 24h avant)	Global ou régional (avec motif)

### 5.3 Règles de quarantaine

- **Compte seul** : désactivation immédiate
- **Branche** : tous les sponsored\_user\_id descendants sont désactivés
- Une quarantaine est **réversible** (historique conservé)

### 5.4 Règles de priorité des horaires

1. Horaire spécial (accordé par admin)
2. Horaire d'équipe (avec rotation)
3. Horaire département
4. Horaire standard du service

## 6. SÉCURITÉ, CONFORMITÉ ET RESPONSABILITÉ

### 6.1 Vérification des conducteurs

Rôle	Documents requis	Validation par
Conducteur privé	Permis (selon politique)	Global coordinator (optionnel)
Conducteur de bus	Permis adapté + assurance	Global coordinator (obligatoire)

### 6.2 Mineurs

- Un compte mineur doit être lié à un compte responsable (parent/tuteur)
- Chaque demande de trajet pour mineur nécessite une validation parentale
- Le conducteur voit un indicateur « mineur – autorisation parentale » sans âge

### 6.3 Accessibilité

- Champ `accessibility_needs` dans `users` (text libre)
- Visible par le conducteur **après** acceptation de l'assignation

### 6.4 Signalements et protection

- Signalement incident → visible immédiatement au coordinateur
- 2 signalements graves confirmés en 90j → suspension automatique du conducteur

## 6.5 Limites de responsabilité

- Écran d'acceptation avant première offre
- Clause annuelle à revalider

## 6.6 RGPD (données personnelles)

Donnée	Conservation	Justification
Localisation domicile	Permanente (tant que compte actif)	Service
Localisation en direct	Supprimée après 7 jours	Coordination
Historique messages	Configurable (défaut 90j)	Litiges
Documents conducteur	Jusqu'à révocation rôle	Conformité
Logs accès localisation	1 an	Audit

## 7. DÉCOUPAGE EN SPRINTS

### Sprint 1 – Fondations (2 semaines)

Tâche	Description
S1.1	Setup monorepo (Turborepo + pnpm)
S1.2	Projet Supabase (local + cloud)
S1.3	Schéma de base (utilisateurs, rôles, parrainage)

S1.4	Authentification (Supabase Auth)
S1.5	RLS de base (propriétaire seul)
S1.6	CI/CD (GitHub Actions + Vercel)

#### Critères d'acceptation :

- Un utilisateur peut s'inscrire
- Un compte est inactif tant qu'il n'est pas parrainé
- Un premier utilisateur (admin) peut être créé en seed

## Sprint 2 – Parrainage et activation (2 semaines)

Tâche	Description
S2.1	Formulaire de parrainage
S2.2	Enregistrement de la chaîne de parrainage
S2.3	Activation manuelle par coordinateur
S2.4	Vue arbre de parrainage (web admin)
S2.5	Quarantaine (compte seul)

#### Critères :

- Un utilisateur actif peut parrainer un inactif
- Le coordinateur global peut activer un compte parrainé
- L'arbre est consultable
- Un compte peut être mis en quarantaine et réactivé

## **Sprint 3 – Conducteur privé – Offre et assignation (2 semaines)**

Tâche	Description
S3.1	Enregistrement véhicule (places)
S3.2	Publication offre (service, segment, départ)
S3.3	Algo simple d'appariement (distance + places)
S3.4	Proposition au conducteur
S3.5	Acceptation / refus conducteur
S3.6	Gel de l'assignation

### **Critères :**

- Un conducteur peut proposer 3 places pour l'aller
- Un passager reçoit une proposition
- Le conducteur accepte → statut « pre-frozen »
- Le conducteur valide → « frozen »

## **Sprint 4 – Passager – Demande et suivi (2 semaines)**

Tâche	Description
S4.1	Capture domicile (Google Places)

S4.2	Formulaire demande trajet (service, segment)
S4.3	Affichage proposition conducteur
S4.4	Acceptation par passager
S4.5	Statut trajet (confirmé, en route, terminé)
S4.6	Messagerie texte + voix (WebSocket)

#### Critères :

- Le passager peut demander un trajet
- Il voit le conducteur proposé
- Il accepte → assignation gelée
- Il peut envoyer un message vocal

## Sprint 5 – Coordination et administration (2 semaines)

Tâche	Description
S5.1	Dashboard demandes non assignées
S5.2	Assignation manuelle (coordinateur)
S5.3	Remplacement conducteur
S5.4	Dégel exceptionnel (avec motif)
S5.5	Création départements / régions
S5.6	Transfert sièges (global_coordinator, région)

**Critères :**

- Un coordinateur voit les demandes sans conducteur
- Il peut assigner manuellement
- Il peut dégeler un trajet (motif loggé)

**Sprint 6 – Bus (mode manuel) (2 semaines)**

Tâche	Description
S6.1	CRUD bus, arrêts, capacité
S6.2	Assignment conducteur de bus
S6.3	Assignment passager à un bus
S6.4	Gel bus (cutoff configurable)
S6.5	Vue conducteur bus (arrêts, passagers)
S6.6	Messagerie bus (broadcast)

**Critères :**

- Un coordinateur crée une ligne de bus (3 arrêts)
- Des passagers y sont assignés
- 24h avant, le plan est gelé
- Le conducteur voit sa liste

## Sprint 7 – Contraintes conducteur bus (1 semaine)

Tâche	Description
S7.1	Capture domicile conducteur bus
S7.2	Définition distance max (départ / retour)
S7.3	Filtrage des assignations infaisables
S7.4	Alerte au coordinateur

### Critères :

- Un conducteur indique « max 10 km autour de chez moi »
- Le système refuse une ligne trop éloignée

## Sprint 8 – Suggestions lignes bus (MILP) (3 semaines)

Tâche	Description
S8.1	Agrégation demande par cluster
S8.2	Génération candidats (via Edge Function)
S8.3	Filtrage contraintes strictes
S8.4	Scoring (couverts, distance conducteur)
S8.5	Comparaison et sélection par coordinateur

### Critères :

- Le système propose 2-3 variantes de ligne
- Les suggestions prennent en compte le domicile conducteur

## **Sprint 9 – Transport public (secours) (2 semaines)**

Tâche	Description
S9.1	Import GTFS (arrêts, horaires)
S9.2	Routage multimodal (église + TC)
S9.3	Affichage « à votre charge »
S9.4	Conseils achat ticket
S9.5	Analytics usage TC par corridor

### **Critères :**

- Un itinéraire peut inclure « bus église → tramway → église »
- Le passager voit que le tram est payant

## **Sprint 10 – Orchestration temps réel (3 semaines)**

Tâche	Description
S10.1	Permission localisation (mobile)
S10.2	Envoi position pendant trajet actif

S10.3	Calcul écart (avance / retard)
S10.4	Vue coordinateur (carte)
S10.5	Logs accès localisation
S10.6	Nettoyage automatique (7 jours)

**Critères :**

- Un conducteur en trajet est visible sur une carte (coordinateur)
- L'utilisateur peut refuser la localisation
- Les logs d'accès sont conservés 1 an

## **Sprint 11 – Finalisation et déploiement pilote (2 semaines)**

Tâche	Description
S11.1	Documentation utilisateur
S11.2	Documentation administrateur
S11.3	Tests de charge (100 utilisateurs)
S11.4	Déploiement production
S11.5	Formation pilote (1 église)

## 8. CRITÈRES D'ACCEPTATION PAR SPRINT

Sprint	Doit avoir (bloquant)	Peut avoir (optionnel)
1	Auth, schéma, RLS, CI	Seed data
2	Parrainage, activation, quarantaine	Arbre graphique
3	Offre, assignation, gel	Notifications email
4	Demande, messagerie vocale	Push notifications
5	Dashboard, assignation manuelle	Export CSV
6	Bus manuel, gel	Broadcast bus
7	Distance max conducteur	Carte contraintes
8	Suggestion MILP	Comparaison visuelle
9	GTFS, multimodal	Analytics
10	Localisation temps réel	Vue carte
11	Documentation, déploiement	Formation

## 9. ENVIRONNEMENTS ET DÉPLOIEMENT

Environnement	URL (exemple)	Utilisation
Development	dev.app.example.com	Développeurs
Staging	staging.app.example.com	Validation interne

## Déploiement production (pilote)

```
bash

Backend (Supabase)
supabase link --project-ref xxx
supabase db push

Admin web
vercel --prod

Mobile
eas build --platform ios --profile production
eas build --platform android --profile production
```

## 10. GLOSSAIRE TECHNIQUE

Terme	Définition
MILP	Mixed-Integer Linear Programming (optimisation linéaire en nombres entiers)
GTFS	General Transit Feed Specification (format horaires transport public)
RLS	Row Level Security (sécurité au niveau ligne dans PostgreSQL)
Edge Function	Fonction serverless exécutée proche de l'utilisateur (Supabase/Deno)
WebSocket	Communication bidirectionnelle temps réel (Supabase Realtime)
PostGIS	Extension géospatiale pour PostgreSQL

**Document prêt pour le développement.**

**Date de validation :** 24 mai 2026

**Prochaines étapes :** Sprint 1 – Fondations

text

---

**## Instructions finales**

1. **\*\*Copiez\*\*** tout le contenu ci-dessus (depuis ````markdown` jusqu'à la fin)
2. **\*\*Collez\*\*** dans un éditeur de texte ou directement dans Google Docs / Word
3. **\*\*Exportez\*\*** au format PDF :
  - **\*\*Google Docs\*\*** : Fichier → Télécharger → PDF
  - **\*\*Word\*\*** : Fichier → Exporter → Créer un PDF/XPS
  - **\*\*VS Code\*\*** : extension "Markdown PDF" → `Ctrl+Shift+P` → "Markdown PDF: Export (pdf)"

Le document fait environ **\*\*15 pages\*\*** une fois converti en PDF.